

Adeil Araújo e Meirivâni Oliveira

Modelagem Computacional
com

Web VPython:
Mecânica

Respostas do Pratique!

Ensino médio

Respostas do Pratique!

Capítulo 1

1. Calcule o valor do produto entre o escalar $a=2$ e o vetor $\vec{b} = \langle 4; -3; 1 \rangle$.

Solução:

$$a \cdot \vec{b} = 2 \cdot \langle 4; -3; 1 \rangle = \langle 8; -6; 2 \rangle$$

```
1 Web VPython 3.2
2
3 a=2
4 b=vec(4,-3,1)
5
6 print(a*b)
```

02. Qual a magnitude do vetor $\vec{f} = \langle 2; 1; 3 \rangle$?

Solução:

$$|\vec{f}| = \sqrt{2^2 + 1^2 + 3^2} = 3,74$$

```
1 Web VPython 3.2
2
3 f=vec(2,1,3)
4
5 print(mag(f))
```

03. Qual o vetor unitário na direção e sentido do vetor $\vec{c} = \langle 3; 3; -2 \rangle$?

Solução:

$$\hat{c} = \frac{\vec{c}}{|\vec{c}|} = \frac{\langle 3; 3; -2 \rangle}{\sqrt{3^2 + 3^2 + (-2)^2}} = \frac{\langle 3; 3; -2 \rangle}{\sqrt{22}} = \langle 0,64; 0,64; -0,42 \rangle$$

```
1 Web VPython 3.2
2
3 c=vec(3,3,-2)
4
5 print(hat(c))
```

04. Fatore o vetor $\vec{e} = \langle 5; 4; -6 \rangle$ em termos do seu vetor unitário, multiplicado pela sua magnitude, $|\vec{e}| \cdot \hat{e}$.

Solução:

$$\vec{e} = |\vec{e}| \cdot \hat{e}$$

Magnitude:

$$|\vec{e}| = \sqrt{5^2 + 4^2 + (-6)^2} = 8,77$$

Vetor unitário:

$$\hat{e} = \frac{\vec{e}}{|\vec{e}|} = \frac{\langle 5; 4; -6 \rangle}{8,77} = \langle 0,57; 0,45; -0,68 \rangle$$

Vetor $\vec{e}$ como produto de um vetor unitário na sua direção e sentido multiplicado por sua magnitude:

$$\vec{e} = |\vec{e}| \cdot \hat{e} = 8,77 \cdot \langle 0,57; 0,45; -0,68 \rangle$$

```
1 Web VPython 3.2
2
3 e=vec(5,4,-6)
4
5 print(mag(e))
6 print(hat(e))
```

05. Para os vetores $\vec{g} = \langle -2; 2; 3 \rangle$ e $\vec{h} = \langle 4; 3; 2 \rangle$, determine:

a) $\vec{g} + \vec{h}$

b) $\vec{g} - \vec{h}$

Solução:

a) $\vec{g} + \vec{h} = \langle (-2) + (4); 2 + 3; 3 + 2 \rangle = \langle 2; 5; 5 \rangle$

b) $\vec{g} - \vec{h} = \langle (-2) - 4; 2 - 3; 3 - 2 \rangle = \langle -6; -1; 1 \rangle$

```
1 Web VPython 3.2
2
3 g=vec(-2,2,3)
4 h=vec(4,3,2)
5
6 print(g+h)
7 print(g-h)
```

06. Dado o vetor $\vec{l} = \langle -2; 3; 4 \rangle$, encontre suas componentes x, y e z.

Solução:

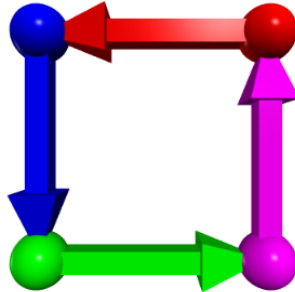
$$\vec{l} = \langle -2; 3; 4 \rangle$$

$$l_x = -2; l_y = 3; l_z = 4$$

```
1 Web VPython 3.2
2
3 l=vec(-2,3,4)
4
5 print(l.x)
6 print(l.y)
7 print(l.z)
```

Capítulo 2

01. Escreva um programa que mostre quatro esferas localizadas, cada uma, em um canto de um quadrado, ligadas por setas no sentido anti-horário. Um arranjo possível é apresentada na figura abaixo.



Solução:

Abaixo apresentamos uma possível solução¹ para o problema.

```
1 Web VPython 3.2
2
3 scene.background=color.white
4
5 bola1=sphere(pos=vec(1,1,0), radius=0.25, color=color.red)
6 bola2=sphere(pos=vec(-1,1,0), radius=0.25, color=color.blue)
7 bola3=sphere(pos=vec(-1,-1,0), radius=0.25, color=color.green)
8 bola4=sphere(pos=vec(1,-1,0), radius=0.25, color=color.magenta)
9 seta1=arrow(pos=bola1.pos, axis=bola2.pos-bola1.pos, shaftwidth=0.2, color=color.red)
10 seta2=arrow(pos=bola2.pos, axis=bola3.pos-bola2.pos, shaftwidth=0.2, color=color.blue)
11 seta3=arrow(pos=bola3.pos, axis=bola4.pos-bola3.pos, shaftwidth=0.2, color=color.green)
12 seta4=arrow(pos=bola4.pos, axis=bola1.pos-bola4.pos, shaftwidth=0.2,
13 color=color.magenta)
```

Na linha 3, definimos a cor de fundo da tela como branco. Nas linhas 5-8, criamos as esferas, atribuindo algumas características como posição (**pos**), raio (**radius**) e cor (**color**).

Nas linhas 9-12, criamos as setas. Elas possuem características diferentes das esferas, como eixo (**axis**), largura do eixo (**Shaftwidth**) além de outras características comuns como posição (**pos**) e cor (**color**).

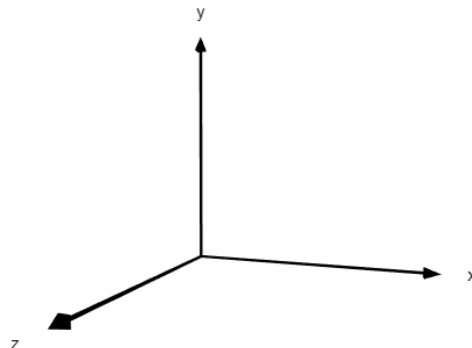
Vamos explicar a construção da **seta1**, linha 9. Isso é suficiente para que você compreenda a construção das demais.

A **seta1**, está localizada na posição **bola1.pos**, isso significa que a sua cauda (origem) está na **bola1** (vermelha). Seu eixo (axis), que é a própria seta, é **bola2.pos-bola1.pos**. Onde **bola2.pos** é a localização da ponta da flecha (extremidade – posição final) e **bola1.pos** é a localização da cauda (origem – posição inicial). O atributo largura do eixo (**Shaftwidth**), é a espessura da seta atribuímos 0.2. Finalmente sua cor é vermelha **color=color.red**.

¹ Para visualizar este programa, acesse o link:

<https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/Cap2E1>

02. Escreva um programa que mostre o sistema de coordenadas 3D apresentado na Figura 5 do capítulo 1. Veja a imagem abaixo.



Solução:

Abaixo apresentamos uma possível solução² para o exercício.

```
1 Web VPython 3.2
2
3 scene.background=color.white
4 x_eixo=arrow(axis=vec(1,0,0), shaftwidth=0.01, color=color.black)
5 y_eixo=arrow(axis=vec(0,1,0), shaftwidth=0.01, color=color.black)
6 z_eixo=arrow(axis=vec(0,0,1), shaftwidth=0.01, color=color.black)
7 label(pos=vec(1.1,0,0), text='x', box=False)
8 label(pos=vec(0,1.1,0), text='y', box=False)
9 label(pos=vec(0,0,1.1), text='z', box=False)
```

Na linha 3, definimos a cor de fundo da tela como branco. Nas linhas 4-6, criamos as setas que representam respectivamente os eixos x, y e z. Um detalhe, na criação das setas não explicitamos o atributo **pos**. Dessa forma, a posição **pos** das setas por padrão será **vec(0,0,0)** quando não especificamos explicitamente este atributo a seta.

Nas linhas 7-9, etiquetamos os eixos com a palavra **label**³. Para etiquetar usamos os atributos **pos**, a localização da etiqueta, **text**, o que você deseja escrever, por exemplo **'x'** para nomear o eixo x e **box**, isso define se seu texto estará dentro de uma caixa **True**, ou não **False**.

² Para visualizar este programa, acesse o link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/eixos>

³ A documentação do VPython está disponível em webvpython.org clicando em "Help".

Capítulo 3

01. Altere a velocidade da bola e veja o que acontece com o comprimento da seta.

Solução:

O comprimento da seta é proporcional ao valor da velocidade.

02. Altere a posição da bola para **pos=vec(10,0,0)** e sua velocidade para **bola.v=vec(-2,0,0)**. Antes de executar tente prever o que irá acontecer.

Solução:

A bola irá se mover no sentido oposto ao original.

03. Altere o intervalo de tempo para **dt=0.1** e o valor entre parênteses de **rate()** para 10. Isso lhe parece melhor?

Solução:

O intervalo de tempo agora é 10 vezes maior e o laço será executado apenas 10 vezes por segundo. A bola ainda se move na mesma velocidade, mas o movimento não parece estar sendo executado em tempo real.

Capítulo 4

01. Modifique o programa para que sua saída inclua o gráfico da componente y da velocidade em função do tempo.

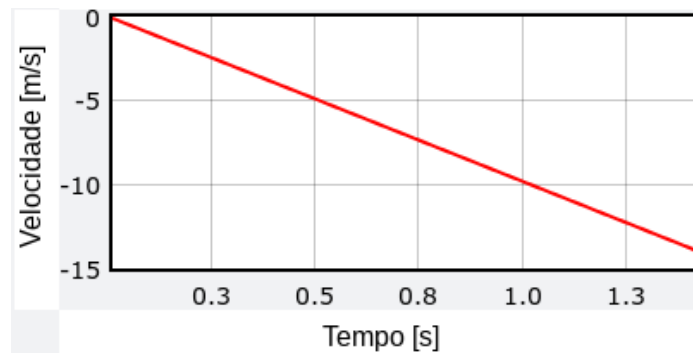
Solução:

Para que a saída do programa apresente o gráfico da componente y da velocidade em função do tempo, precisamos incluir no programa⁴ as linhas 21, 22 e 31, como mostra a figura abaixo. O processo é o mesmo que utilizamos para construir o gráfico da posição em função do tempo.

```
1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.title="Queda Livre"
5 scene.background=vec(0.6, 0.6, 0.6)
6
7 #2. Criação dos objetos
8 bola=sphere(pos=vec(0,10,0), radius=1, color=color.red)
9 piso=box(pos=vec(0,-1,0), size=vec(40,0.3,10), texture=textures.wood)
10
11 #3. Condições iniciais
12 bola.a=vec(0,-9.8,0) # aceleração da gravidade 9.8 m/s^2
13 bola.v=vec(0,0,0)    # velocidade da bola na direção y
14 t=0                  # tempo inicial
15 dt=0.01              # intervalo de tempo
16
17 #4. Configuração do gráfico da posição
18 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]", width=400, height=200)
19 yt=gcurve(color=color.blue)
20
21 Grafico_vt=graph(xtitle="Tempo [s]", ytitle="Velocidade [m/s]", width=400, height=200)
22 vt=gcurve(color=color.red)
23
24 #5. Laço while - Movendo o objeto
25 while (bola.pos.y-bola.radius > piso.pos.y+0.5*piso.size.y):
26     rate(100)
27     bola.v+=bola.a*dt
28     bola.pos+=bola.v*dt
29     t+=dt
30     yt.plot(t,bola.pos.y)
31     vt.plot(t,bola.v.y)
32
33 print(t)
```

A saída mostra o gráfico de uma função linear, como mostra a figura abaixo.

⁴ Para visualizar este programa, acesse o link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/QuedaLivreGrVT>



02. Modifique o programa estudado, neste capítulo, para modelar a queda de uma esfera de massa m que cai no ar sob a ação da aceleração gravitacional e de uma força de arrasto proporcional a v^2 , conforme a seguinte expressão:

$$F_{ar} = b \cdot v^2$$

Onde b (kg/m ou $\text{N/m}^2 \cdot \text{s}^{-2}$), é uma constante que depende das características do objeto e das propriedades do fluido e é obtido pela seguinte expressão:

$$b = \frac{1}{2} c_d \cdot A \cdot \rho$$

Onde c_d é um parâmetro determinado de forma experimental, conhecido como coeficiente de arrasto, ρ é a massa específica do ar (massa por unidade de volume) e A é a área transversal exposta à corrente de ar.

Construa os gráficos da componente y da posição e da componente y da velocidade em função do tempo, assim como a velocidade terminal da esfera.

Obs.: Atente para o fato de que não é possível elevar um vetor ao quadrado; você tem que elevar ao quadrado a magnitude do vetor.

Solução:

O Programa⁵ criado é muito parecido com o que foi estudado no capítulo, como pode ser observado abaixo.

```

1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.title="Queda Livre"
5 scene.background=vec(0.6, 0.6, 0.6)
6
7 #2. Criação dos objetos
8 bola=sphere(pos=vec(0,10,0), radius=1, color=color.red)
9 piso=box(pos=vec(0,-1,0), size=vec(40,0.3,10), texture=textures.wood)
10
11 #3. Condições iniciais
12 bola.m=0.5 # massa da bola em kg
13 g=9.8 # aceleração gravitacional em m/s**2

```

⁵ Para visualizar este programa, acesse o

link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/QuedaArraste>

```

14 b=0.2 # coeficiente de arraste em kg/m
15 bola.v=vec(0,0,0) # velocidade da bola na direção y
16 t=0 # tempo inicial
17 dt=0.01 # intervalo de tempo
18
19 #4. Configuração do gráfico da posição
20 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]",width=400, height=200)
21 yt=gcurve(color=color.blue)
22
23 Grafico_vt=graph(xtitle="Tempo [s]", ytitle="Velocidade [m/s]", width=400, height=200)
24 vt=gcurve(color=color.red)
25
26 #5. Laço while - Movendo o objeto
27 while (bola.pos.y-bola.radius > piso.pos.y+0.5*piso.size.y):
28     rate(100)
29     f=b*(mag2(bola.v))*-hat(bola.v)
30     bola.a=vec(0,-g,0)+f/bola.m
31     bola.v+=bola.a*dt
32     bola.pos+=bola.v*dt
33     t+=dt
34     yt.plot(t,bola.pos.y)
35     vt.plot(t,bola.v.y)
36
37 print(t)

```

Vamos ver o que foi modificado.

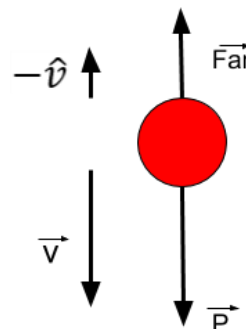
Primeiro, inserimos na seção “*Condições iniciais*”, um valor para a massa da bola, o valor de g (aceleração gravitacional) e o coeficiente de arraste, linhas 12,13 e 14, respectivamente.

Na seção “*Configuração do gráfico da posição e da velocidade*”, incluímos as linhas, 23 e 24 para criação do gráfico da velocidade.

Porém, a principal mudança se encontra na seção “*Laço while – Movendo objeto*”. Já que estamos considerando a resistência do ar, ela é adicionada nas linhas 29 e 30.

Na linha 29, incluímos a força de arraste “ $f=b*(mag2(bola.v))*-hat(bola.v)$ ”. Fatoramos a força de arraste em termos de sua magnitude, multiplicada por um vetor unitário em sua direção e sentido.

$$\vec{F}_{ar} = b \cdot v^2 \cdot (-\hat{v})$$



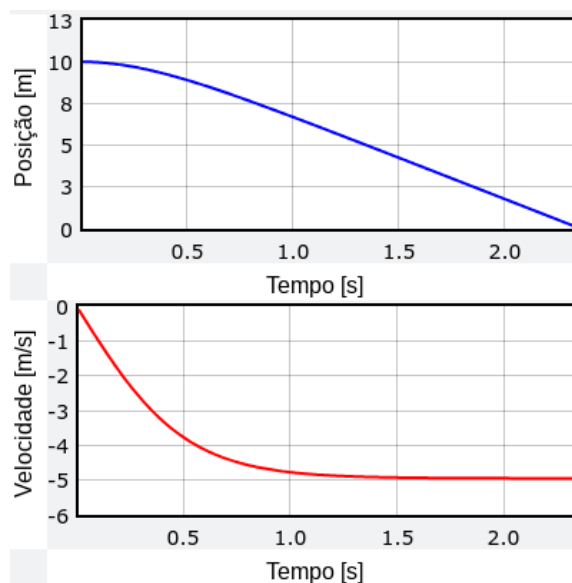
O termo “ $(mag2(bola.v))$ ” é usado para calcular o quadrado da magnitude do vetor velocidade. A função *mag()* calcula a magnitude de um vetor, enquanto a

função *mag2()* calcula o quadrado dessa magnitude e termo “*-hat(bola.v)*” é usado para calcular o vetor unitário na direção e sentido da força de arraste.

Na linha 30, calculamos o vetor aceleração da bola. Aqui a bola está sujeita a duas forças: a gravitacional e de arraste. Desta forma a aceleração resultante é a soma vetorial “*bola.a= vec(0,-g,0)+f/bola.m*”. Onde o termo “*vec(0,-g,0)*” é o vetor aceleração gravitacional e “*f/bola.m*” é a aceleração devido a força de arraste.

Finalmente, na linha 35 incluímos o trecho de código responsável por adicionar um ponto ao gráfico da velocidade em função do tempo a cada execução do laço.

Ao executar o programa obtemos além da simulação da queda da bola os gráficos da componente y posição e da componente y da velocidade em função do tempo. Como pode ser observado abaixo.



Na figura observamos que a velocidade aumenta em módulo com o passar do tempo, de forma não linear até atingir um valor limite, conhecido como velocidade terminal. No gráfico, é possível observar que esse valor limite possui módulo igual a 5 m/s.

Capítulo 5

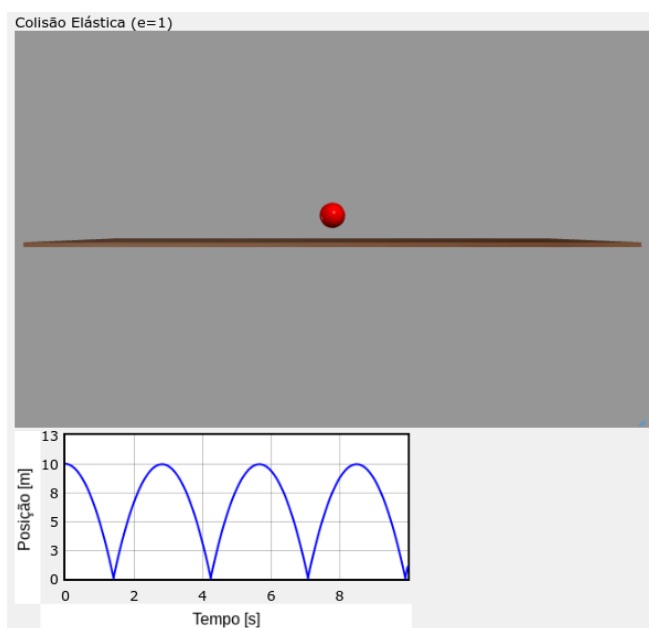
01. Modifique o programa apresentado no capítulo, para que haja:

- a) Uma colisão elástica⁶ da bola com o piso;
- b) Uma colisão inelástica⁷ da bola com o piso.

Analisar os gráficos da componente y da posição em função do tempo em cada situação.

Solução:

Para resolver tanto o item (a) como o (b), basta modificar o valor do coeficiente de restituição e , para 1, caso da colisão elástica, e posteriormente para zero, caso da colisão inelástica. Observe os gráficos da posição pelo tempo para cada caso abaixo.



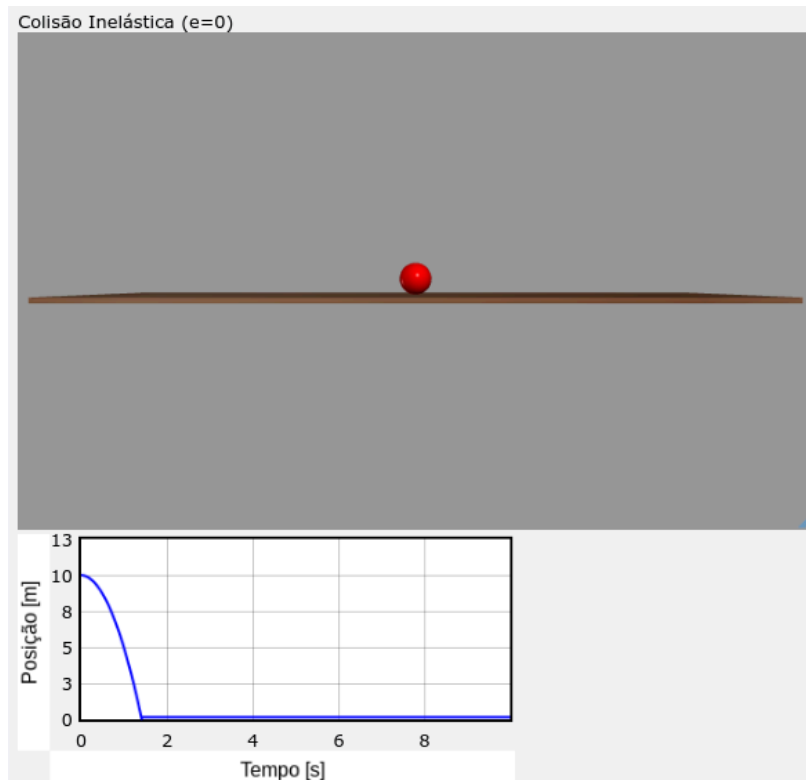
Resposta (a)

⁶ Para visualizar este programa, acesse o link:

<https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/ColisaoElastica>

⁷ Para visualizar este programa, acesse o link

<https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/ColisaoInelastica>



Resposta (b)

02. Modifique o programa para que ele forneça também o gráfico da componente y da velocidade em função do tempo.

Solução:

Uma possível solução é apresentada no programa⁸ abaixo.

```

1  Web VPython 3.2
2
3  #1. Configuração da tela
4  scene.title="Bola saltitante"
5  scene.background=vec(0.6, 0.6, 0.6)
6
7  #2. Criação dos objetos
8  bola=sphere(pos=vec(0,10,0), radius=1, color=color.red)
9  piso=box(pos=vec(0,-1,0), size=vec(40, 0.3, 10), texture=textures.wood)
10
11 #3. Condições iniciais
12 bola.a=vec(0,-9.8,0) # aceleração da gravidade 9.8 m/s^2
13 bola.v=vec(0,0,0)    # velocidade da bola na direção y
14 t=0                  # tempo
15 dt=0.01              # intervalo de tempo
16 e=0.74               # coeficiente de restituição
17

```

⁸ Para visualizar este programa, acesse o link:

<https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/BolaSaltitanteGRVT>

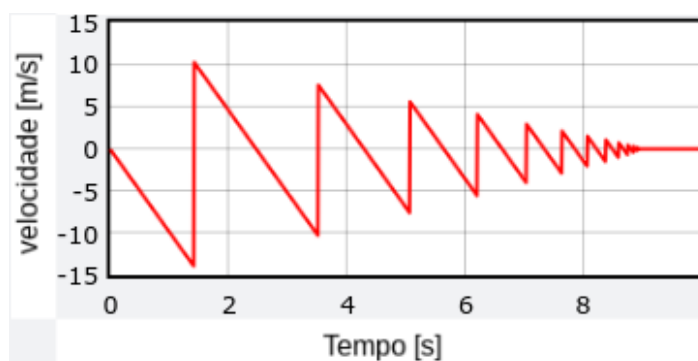
```

18 #4. Configuração do gráfico da posição
19 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]",width=400, height=200)
20 yt=gcurve(color=color.blue)
21
22 Grafico_vt=graph( xtitle="Tempo [s]", ytitle="velocidade [m/s]",width=400, height=200)
23 vt=gcurve(color=color.red)
24
25 #5. Laço While - Movendo o objeto
26 while t<10:
27     rate(100)
28     if (bola.pos.y - bola.radius < piso.pos.y + 0.5*piso.size.y):
29         bola.pos.y=piso.pos.y+0.5*piso.size.y+bola.radius
30         bola.v.y=abs(bola.v.y)*e
31
32     bola.v+=bola.a*dt
33     bola.pos+=bola.v*dt
34     attach_arrow(bola,"v", color=color.black, shaftwidth=0.25)
35     attach_arrow(bola,"a", color=color.blue, shaftwidth=0.25)
36     yt.plot(t,bola.pos.y)
37     vt.plot(t,bola.v.y)
38     t+=dt

```

Para obtermos o gráfico da velocidade em função do tempo, incluímos as linhas 22 e 23 na seção 4 “Configuração do gráfico da posição e da velocidade” e a linha 37 na seção “Laço while – movendo o objeto”.

O gráfico da velocidade em função do tempo pode ser observado na figura abaixo.



Como se pode observar no gráfico de v_y em função do tempo, ocorre uma mudança de sentido da velocidade da bola na vertical (v_y), que inicialmente era zero e que depois aumentou na direção negativa devido à ação descendente da aceleração da gravidade. Uma vez que ela toca o solo, inverte-se a direção de sua velocidade que tem sua intensidade reduzida por um fator de aproximadamente 0,74 (coeficiente de restituição).

Você pode também incluir o gráfico da velocidade horizontal (v_x) em função do tempo. Você perceberá que ela permanece constante. Isso porque a bola é acelerada pela gravidade na direção y , mas na direção x , não.

Capítulo 6

01. Inclua ao final do código estudado neste capítulo a linha de código:

```
print("O alcance foi: ",bola.pos.x+20,"metros")
```

Para descobrir o alcance da bola.

Solução:

A expressão “*bola.pos.x+20*” é o alcance horizontal da bola. Sua posição final “*bola.pos.x*”, subtraído de sua posição inicial “*-20 m*”.

02. Modifique o programa para que ele forneça também o gráfico da componente x posição em função do tempo.

Solução:

Veja o programa⁹ abaixo.

```
1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.title="Lançamento de Projéteis"
5 scene.background=vec(0.6, 0.6, 0.6)
6
7 #2. Criação dos objetos
8 bola=sphere(pos=vec(-20,0,0), radius=1, color=color.red, make_trail=True)
9 piso=box(pos=vec(0,-1,0), size=vec(40, 0.3, 10), texture=textures.wood)
10
11 #3. Condições iniciais
12 bola.a=vec(0,-9.8,0) # aceleração da gravidade 9.8 m/s^2
13 v0=20 # velocidade inicial
14 theta=radians(30) # ângulo de lançamento
15 bola.v=v0*vec(cos(theta), sin(theta), 0) # velocidade da bola
16 t=0 # tempo inicial
17 dt=0.01 # intervalo de tempo
18
19 #4. Configuração do gráfico da posição
20 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]", width=400, height=200)
21 yt=gcurve(color=color.blue)
22
23 Grafico_xt=graph(xtitle="Tempo [s]", ytitle="Posição [m]",width=400, height=200)
24 xt=gcurve(color=color.blue)
25
26 #5. Laço while - Movendo o objeto
27 while (bola.pos.y - bola.radius >= piso.pos.y): # outra opção: while (bola.pos.y)>=0:
28     rate(100)
29     bola.v+=bola.a*dt
30     bola.pos+=bola.v*dt
31
```

⁹ Para visualizar este programa, acesse o

link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/alcance>

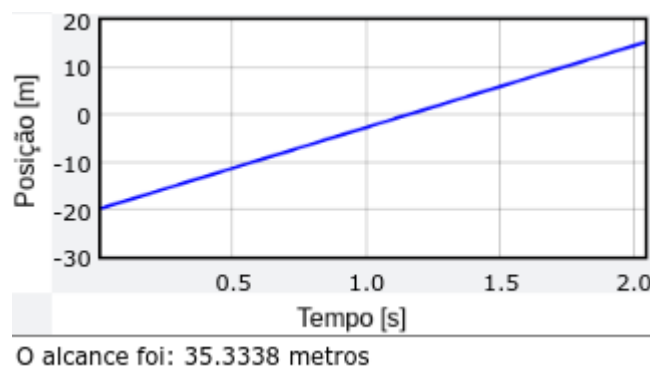
```

32 attach_arrow(bola,"v", color=color.black, shaftwidth=0.25)
33 attach_arrow(bola,"a", color=color.blue, shaftwidth=0.25)
34
35 t+=dt
36 yt.plot(t,bola.pos.y)
37 xt.plot(t,bola.pos.x)
38
39 print("O alcance foi:",bola.pos.x+20,"metros")

```

Para incluir o gráfico da posição horizontal em função do tempo, incluímos as linhas 23 e 24 na seção “Configuração dos gráficos da posição vertical e horizontal” e a linha 37 na seção “Laço while - Movendo o objeto”.

Observe o resultado na figura abaixo.



04. Modifique o programa estudado, neste capítulo, para modelar o lançamento oblíquo de uma esfera de massa m que se move sob a ação de uma força de arrasto proporcional a v^2 ,

$$F_{ar} = b \cdot v^2$$

Solução:

O programa que apresentaremos é bastante semelhante ao que escrevemos no capítulo 4 para uma esfera que caia em queda livre e experimentava uma força de arraste também proporcional a v^2 .

Um possível programa¹⁰ que modele este problema é exibido abaixo.

```

1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.title="Lançamento de Projéteis com resistência do ar"
5 scene.background=vec(0.6, 0.6, 0.6)
6
7 #2. Criação dos objetos
8 bola=sphere(pos=vec(-20,0,0), radius=.5, color=color.red, make_trail=True)
9 piso=box(pos=vec(0,-1,0), size=vec(40, 0.3 , 10), texture=textures.wood)
10
11 #3. Condições iniciais
12 bola.m=2 # massa em kg

```

¹⁰ Para visualizar este programa, acesse o

link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/ProjeteisArraste>

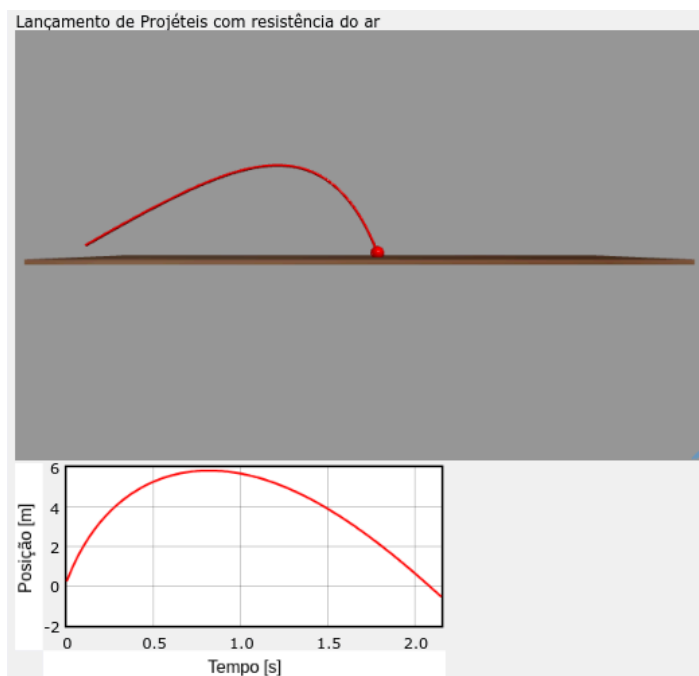

```

13 g=9.8 # aceleração gravitacional em m/s**2
14 v0=50 # velocidade inicial em m/s
15 b=0.2 # coeficiente de arraste em kg/m
16 theta=radians(30) # ângulo de lançamento
17 bola.v=v0*vec(cos(theta), sin(theta), 0) # velocidade da bola
18 t=0 # tempo inicial
19 dt=0.01 # intervalo de tempo
20
21 #4. Configuração do gráfico da posição
22 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]", width=400, height=200)
23 yt=gcurve(color=color.red)
24
25 #5. Laço while - Movendo o objeto
26 while (bola.pos.y - bola.radius >= piso.pos.y):
27     rate(100)
28     f=b*(mag2(bola.v))*-hat(bola.v)
29     bola.a=vec(0,-g,0)+f/bola.m
30     bola.v+=bola.a*dt
31     bola.pos+=bola.v*dt
32
33     yt.plot(t,bola.pos.y)
34
35     t+=dt

```

Repare nas linhas 28 e 29, são as linhas onde incluímos o vetor força de arraste e o vetor aceleração. Elas são necessárias para incluir a resistência do ar.

A saída do programa é apresentada na figura abaixo.

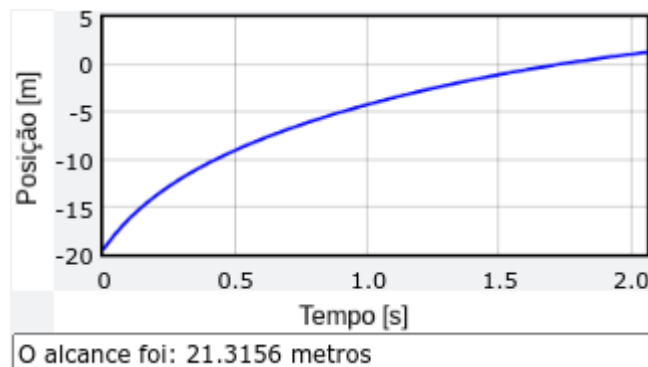


Observe forma da trajetória, ela se desvia da parábola. Observe que o ponto mais alto da trajetória não está no meio do caminho, e a descida é mais íngreme do que a subida.

05. Execute do exercício 4 para diferentes ângulos de lançamento e determine o ângulo que lhe dá o maior alcance. Sem arrasto de ar sabemos que este ângulo é de 45 graus.

Solução:

O maior alcance¹¹ foi encontrado para um ângulo de lançamento de 28°. Porém, tivemos valores de alcance próximos no intervalo entre 28° e 30°. A figura abaixo mostra um gráfico da posição na direção x em função do tempo para um ângulo de lançamento de 28°.



Para mais informações, leia o artigo: “*Maximum projectile range with drag and lift, with particular application to golf*”¹². No artigo proposto para leitura, o autor encontrou para um arrasto linear, alcances máximos entre 30° e 35° e para um arraste quadrático um alcance máximo para um ângulo de 35°.

06. Escreva um programa que forneça como saída um modelo computacional que apresente duas bolas idênticas, lançadas de um mesmo ângulo inicial e em movimento oblíquo. Sendo que uma delas viaja sob a ação da aceleração gravitacional e de uma força de arraste proporcional a v^2 , enquanto a segunda viaja apenas sob a ação da aceleração gravitacional.

Solução:

Um possível código¹³ que modela este problema é muito parecido com o código do exercício anterior. Antes de prosseguir, compare o dois.

```
1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.title="Lançamento de Projéteis com resistência e sem resistência do ar"
5 scene.background=vec(0.6, 0.6, 0.6)
```

¹¹ Para visualizar este programa, acesse o

link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/RE5CAP6>

¹² Disponível em: <http://www2.odu.edu/~agodunov/teaching/phys420/files/Erlichson.pdf>

¹³ Para visualizar este programa, acesse o link:

<https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/ProjeteisArraste2>

```

6
7 #2. Criação dos objetos
8 bola=sphere(pos=vec(-20,0,0), radius=1, color=color.red, make_trail=True)
9 bola2=sphere(pos=vec(-20,0,0), radius=1, color=color.green, make_trail=True)
10 piso=box(pos=vec(0,-1,0), size=vec(40, 0.3 , 10), texture=textures.wood)
11
12 #3. Condições iniciais
13 bola.m=2
14 g=9.8
15 v0=20
16 b=0.2
17 bola2.a=vec(0,-9.8,0)
18 theta=radians(30)
19 bola.v=v0*vec(cos(theta), sin(theta), 0)
20 bola2.v=v0*vec(cos(theta), sin(theta), 0)
21 t=0
22 dt=0.01
23
24 #4. Configuração do gráfico da posição
25 Grafico_xt=graph(xtitle="Tempo [s]", ytitle="Posição [m]",width=400, height=200)
26 xt=gcurve(color=color.red)
27
28 Grafico_xt2=graph(xtitle="Tempo [s]", ytitle="Posição [m]",width=400, height=200)
29 xt2=gcurve(color=color.green)
30
31 #5. Laço while - Movendo o objeto
32 while(bola.pos.y-bola.radius>=piso.pos.y) or (bola2.pos.y-bola2.radius)>=piso.pos.y:
33     rate(100)
34     f=b*(mag2(bola.v))*-hat(bola.v)
35     bola.a=vec(0,-g,0)+f/bola.m
36
37     bola.v+=bola.a*dt
38     bola.pos+=bola.v*dt
39
40     bola2.v+=bola2.a*dt
41     bola2.pos+=bola2.v*dt
42
43     if bola.pos.y - piso.pos.y <= bola.radius:
44         bola.v=vec(0, 0, 0)
45
46     xt.plot(t,bola.pos.y)
47     xt2.plot(t,bola2.pos.y)
48
49     t+=dt

```

Vamos as diferenças.

Na seção “*criação de objetos*”, criamos uma segunda bola, que chamamos de bola2. Também modificamos os valores de raio e as diferenciamos apenas pelas cores.

Na seção “*condições iniciais*”, definimos a aceleração da bola2 (linha 17) e sua velocidade inicial (linha 20). A bola2 está sujeita apenas à aceleração gravitacional.

Na seção “*Configuração do gráfico da posição*” incluímos os comandos para que fosse possível traçar o gráfico da bola2.

As maiores diferenças encontram-se no laço *while*. Primeiramente na condição do laço “(bola.pos.y - bola.radius >= piso.pos.y) or (bola2.pos.y - bola2.radius) >=

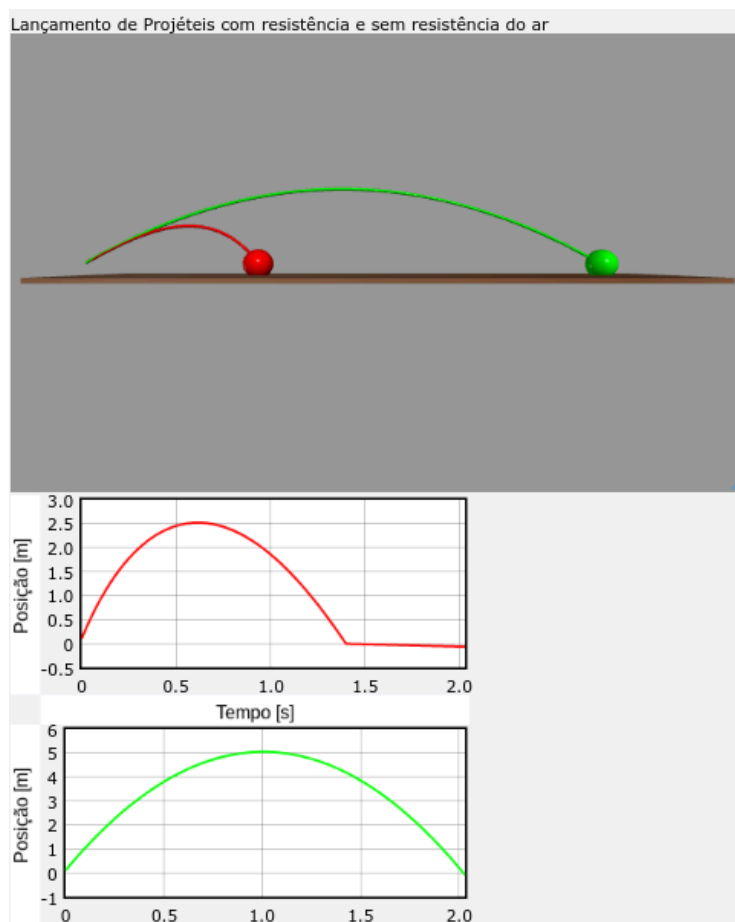
piso.pos.y:”. Esta condição diz que enquanto a bola ou a bola2 não tocar o piso (solo), faça tudo o que estiver endentado.

Nas linhas 40, 41 acrescentamos as equações de atualização da velocidade e da posição para a bola2.

Nas linhas 43, 44 aparece a condição “if”: “bola.pos.y - piso.pos.y <= bola.radius: bola.v = vec(0, 0, 0)”, que diz que se a bola tocar o solo, faça com que ela entre em repouso (velocidade vetorial igual a zero). Comente estas linhas de código para ver o que acontece (basta colocar o símbolo “#”, na frente de cada uma das linhas).

Finalmente inserimos na linha 47, incluímos o trecho de código responsável por adicionar um ponto ao gráfico da posição em função do tempo a cada execução do laço.

O resultado da execução é apresentado abaixo.



A bola (vermelha) sob ação da aceleração gravitacional e da força de arraste, viajou por quase 1,4 segundos enquanto a bola que estava apenas sob a ação da força gravitacional viajou por 2s.

Capítulo 7

01. Modifique o programa para que ele forneça também os gráficos da componente y da velocidade e da componente y da aceleração em função do tempo.

Solução:

Uma possível solução é apresentada no programa¹⁴ abaixo.

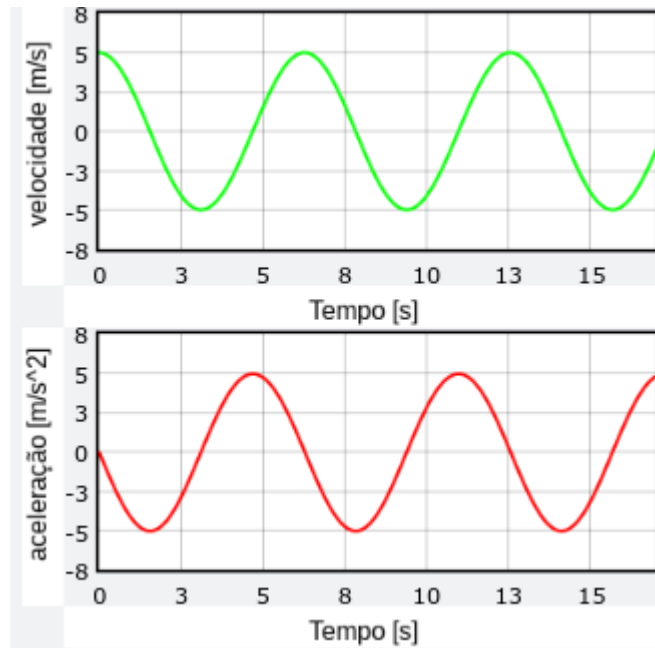
```
1 Web VPython 3.2
2
3 #1.Configuração da tela
4 scene.title="Movimento circular uniforme"
5
6 #2. Criação dos objetos
7 bola=sphere(pos=vec(5, 0, 0), radius=0.5, color=color.red, make_trail=True)
8
9 #3. Condições iniciais
10 R=5 # Raio do movimento circular
11 bola.v=vec(0, 10, 0) # Velocidade em m/s
12 t=0 # Tempo inicial
13 dt=0.01 # Tempo entre cada quadro da animação
14
15 #4. Configuração do gráfico da posição
16 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]", width=400, height=200)
17 yt=gcurve(color=color.blue)
18
19 Grafico_vyt=graph(xtitle="Tempo [s]", ytitle="velocidade [m/s]",width=400, height=200)
20 vyt=gcurve(color=color.green)
21
22 Grafico_ayt=graph(xtitle="Tempo [s]", ytitle="aceleração [m/s^2]",width=400, height=200)
23 ayt=gcurve(color=color.red)
24
25 #5. Laço While - Movendo o objeto
26 while (True):
27     rate(100)
28     r=bola.pos-vec(0,0,0)
29     bola.a=(bola.v.mag2/R)*-hat(r)
30
31     bola.v+=bola.a*dt
32     bola.pos+=bola.v*dt
33
34     attach_arrow(bola,"v", color=color.magenta, shaftwidth=0.25)
35     attach_arrow(bola,"a", color=color.yellow, shaftwidth=0.25)
36
37     yt.plot(t,bola.pos.y)
38     vyt.plot(t,bola.v.y)
39     ayt.plot(t,bola.a.y)
40
41     t+=dt
```

¹⁴ Para visualizar este programa, acesse o

link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/MovimentoCircular3>

Para obtermos o gráfico da componente y da velocidade em função do tempo, incluímos as linhas 19-20 e para obter o gráfico da componente y da aceleração em função do tempo, incluímos as linhas 22-23 na seção “*Configuração do gráfico da posição e da velocidade*” e as linhas 38-39 na seção “*Laço while – movendo o objeto*”.

O gráfico da componente y da velocidade e da componente y da aceleração em função do tempo pode ser observado na figura abaixo.



Capítulo 8

01. Modifique o programa para que ele forneça também o gráfico da componente y da velocidade em função do tempo.

Solução:

Uma possível solução é apresentada no programa¹⁵ abaixo.

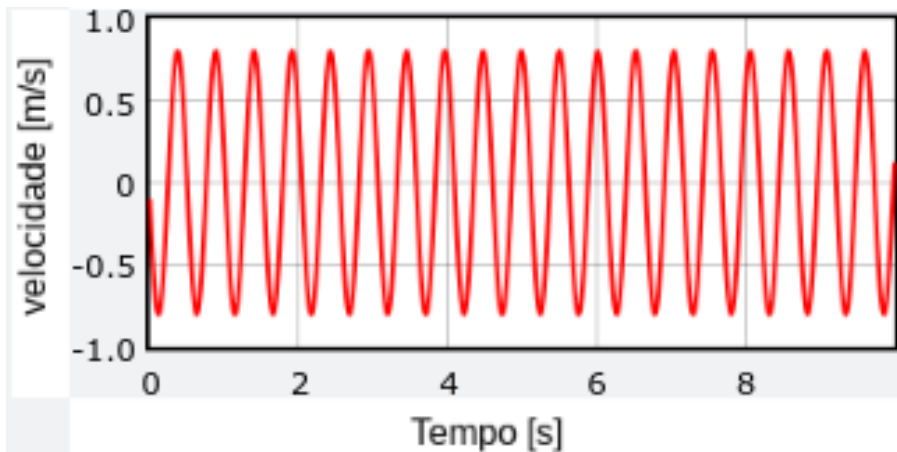
```
1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.background=color.white
5
6 #2. Criação dos objetos
7 teto=box(pos=vec(0,0,0), size=vec(0.1,0.005, 0.1), texture=textures.wood)
8 bola=sphere(pos=vec(0,-0.1,0), radius=0.02, color=color.red)
9 mola=helix(pos=teto.pos, axis=bola.pos-teto.pos, coils=15, radius=0.008, thickness=0.003, color=color.green)
10
11 #3. Condições iniciais
12 scene.center=bola.pos # Colocando a bola no centro da tela
13 bola.v=vec(0,0,0) # Velocidade inicial em m/s
14 L0=0.1 # Comprimento da mola relaxada
15 k=15 # Constante elástica em N/m
16 bola.m=0.1 # Massa da bola em kg
17 g=vec(0,-9.8,0) # Aceleração gravitacional
18 t=0 # Tempo inicial
19 dt=0.01 # Intervalo de tempo entre cada quadro
20
21 #4. Configuração do gráfico da posição
22 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]", width=400, height=200)
23 yt=gcurve(color=color.blue)
24
25 Grafico_vt=graph(xtitle="Tempo [s]", ytitle="velocidade [m/s]", width=400, height=200)
26 vt=gcurve(color=color.red)
27
28 #5. Laço While - Movendo o objeto
29 while t<10:
30     rate(100)
31     L=bola.pos-teto.pos
32     Fs=-k*(mag(L)-L0)*hat(L)
33     F= Fs+bola.m*g
34
35     bola.v+=F/bola.m*dt
36     bola.pos+=bola.v*dt
37     mola.axis=bola.pos-teto.pos
38     yt.plot(t,bola.pos.y)
39     vt.plot(t,bola.v.y)
40     t+=dt
```

Para obtermos o gráfico da velocidade em função do tempo, incluímos as linhas 25 e 26 na seção 4 “Configuração do gráfico da posição e da velocidade” e a linha 39 na seção “Laço while – movendo o objeto”.

O gráfico da velocidade em função do tempo pode ser observado na figura abaixo.

¹⁵ Para visualizar este programa, acesse o

link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/ForcaElasticaGRVT>



Um dado que pode vir a ser explorado é a distinção entre a função seno e a função cosseno. O gráfico da posição em função do tempo é uma função seno, enquanto o gráfico da velocidade em função do tempo é uma função cosseno.

02. Adicione uma força de arrasto ao programa do exercício anterior.

Solução:

```

1 Web VPython 3.2
2
3 #1. Configuração da tela
4 scene.background=color.white
5
6 #2. Criação dos objetos
7 teto=box(pos=vec(0,0,0), size=vec(0.1,0.005, 0.1), texture=textures.wood)
8 bola=sphere(pos=vec(0,-0.1,0), radius=0.02, color=color.red)
9 mola=helix(pos=teto.pos, axis=bola.pos-teto.pos, coils=15, radius=0.008, thickness=0.003, color=color.green)
10
11 #3. Condições iniciais
12 scene.center=bola.pos # Colocando a bola no centro da tela
13 bola.v=vec(0,0,0) # Velocidade inicial em m/s
14 L0=0.1 # Comprimento da mola relaxada
15 k=15 # Constante elástica em N/m
16 b=0.1 # coeficiente de arraste em kg/s
17 bola.m=0.1 # Massa da bola em kg
18 g=vec(0,-9.8,0) # Aceleração gravitacional
19 t=0 # Tempo inicial
20 dt=0.01 # Intervalo de tempo entre cada quadro
21
22 #4. Configuração do gráfico da posição
23 Grafico_yt=graph(xtitle="Tempo [s]", ytitle="Posição [m]", width=400, height=200)
24 yt=gcurve(color=color.blue)
25
26 Grafico_vt=graph(xtitle="Tempo [s]", ytitle="velocidade [m/s]", width=400, height=200)
27 vt=gcurve(color=color.red)
28
29 #5. Laço While - Movendo o objeto
30 while t<10:
31     rate(100)
32     f=-b*bola.v
33     L=bola.pos-teto.pos
34     Fs=-k*(mag(L)-L0)*hat(L)
35     F=Fs+bola.m*g+f
36
37     bola.v+=F/bola.m*dt
38     bola.pos+=bola.v*dt
39     mola.axis=bola.pos-teto.pos
40     yt.plot(t,bola.pos.y)

```



```

41 vt.plot(t,bola.v.y)
42 t+=dt

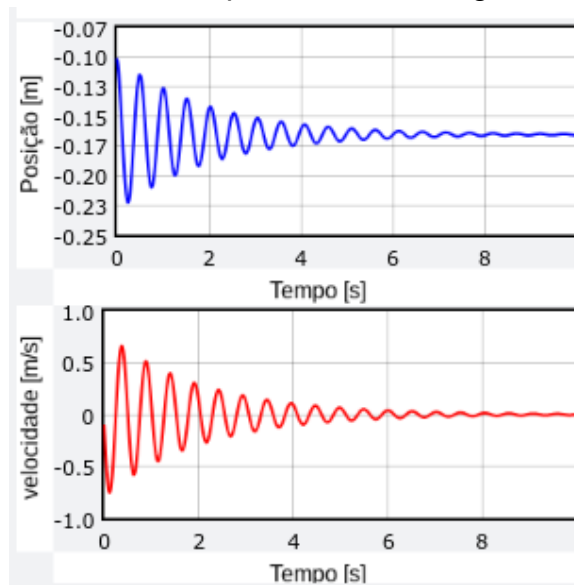
```

Uma possível solução é apresentada no programa¹⁶ abaixo.

As alterações em relação ao programa do exercício anterior estão presentes nas linhas:

- 16, onde inserimos o coeficiente de arreste b , cuja unidade no SI é kg/s;
- 32, onde inserimos a força de arraste. O sinal negativo na equação “ $f = -b \cdot bola.v$ ”, se deve ao fato da força de arraste ser oposta a velocidade da bola. Aqui estamos supondo que a força de arraste é proporcional a velocidade da bola;
- 35, onde adicionamos a força de arraste à força resultante.

Os gráficos do movimento são apresentados na figura abaixo.



¹⁶ Para visualizar este programa, acesse o link: <https://www.glowscript.org/#/user/adeil.fisica/folder/MyPrograms/program/ForcaElasticaArraste>